WEEK – 12

Experiment No. 12: Multithreading

Title

Seat Reservation System Using Multithreading and Synchronization

Objective / Aim

To write a Java program to implement a seat reservation system using multithreading and synchronization. The program allows multiple person threads to book seats while ensuring that only one thread can access the reservation method at a time.

Problem Statement

Implement a reservation system that allows multiple persons to book seats.

Define a `reserve()` method with an initial availability of 10 seats. Create multiple person threads to book seats.

At any time, only one person thread should be allowed to access the `reserve()` method. Use the `synchronized` keyword to prevent multiple threads from accessing the reservation method simultaneously.

The reservation system should check the available seats before confirming a booking.

Concepts Used

Concept	Description
Multithreading	Executing multiple threads concurrently.
Thread	A lightweight unit of execution within a program.
Thread Class	Used to create and manage threads.
`start()`	Starts the execution of a thread.
`run()`	Contains the task performed by a thread.
Synchronization	Controls access to shared resources by multiple threads.

Concept	Description
`synchronized` Keyword	Ensures that only one thread at a time can execute a synchronized instance method on the same object.
Shared Resource	The available seat count shared among all person threads.
Constructor	Initializes the person name and number of seats requested.

Classes Used

1. **Reservation** – Maintains the available seats and defines the synchronized `reserve()` method.
2. **Person** – Extends the `Thread` class and requests seats through the reservation system.
3. **ReservationDemo** – Contains the `main()` method and creates multiple person threads.

Algorithm / Logic

1. Start the program.
2. Create a `Reservation` class with an initial availability of 10 seats.
3. Define a synchronized `reserve()` method.
4. Pass the person's name and requested number of seats to the method.
5. Display the person's entry message.
6. Display the available seats and requested seats.
7. Check whether sufficient seats are available.
8. If sufficient seats are available, reserve the requested seats and update the available seat count.
9. Otherwise, display a message indicating that the requested seats are unavailable.
10. Display the person's leaving message.
11. Create a `Person` class that extends the `Thread` class.
12. Override the `run()` method to call the synchronized `reserve()` method.
13. Create three person threads with requests for 5, 2, and 4 seats, respectively.
14. Start the threads in the required order.
15. Use `join()` to wait for each thread to finish before starting the next one, so the output follows the required sequence.
16. Stop the program.

Source Code

File Name: ReservationDemo.java

```
class Reservation {

    private int availableSeats = 10;

    public synchronized void reserve(String personName,
                                     int requestedSeats) {

        System.out.println(personName + " entered.");

        System.out.println("Available seats: " + availableSeats
                + " Requested seats: " + requestedSeats);

        if (requestedSeats <= availableSeats) {

            System.out.println("Seat Available. Reserve now :-)");

            availableSeats -= requestedSeats;

            System.out.println(requestedSeats + " seats reserved.");

        } else {

            System.out.println("Requested seats not available :-)");

        }

        System.out.println(personName + " leaving.");
        System.out.println("-----------------------------------------------");
    }
}

class Person extends Thread {

    private Reservation reservation;
    private int requestedSeats;

    public Person(String name, Reservation reservation,
                  int requestedSeats) {

        super(name);
        this.reservation = reservation;
        this.requestedSeats = requestedSeats;
    }

    @Override
    public void run() {

        reservation.reserve(getName(), requestedSeats);
    }
}

public class ReservationDemo {

    public static void main(String[] args)
            throws InterruptedException {

        Reservation reservation = new Reservation();
```

```
        Person p1 = new Person("Person-1", reservation, 5);
        Person p2 = new Person("Person-2", reservation, 2);
        Person p3 = new Person("Person-3", reservation, 4);

        p1.start();
        p1.join();

        p2.start();
        p2.join();

        p3.start();
        p3.join();
    }
}
```

Explanation of the Program

1. Reservation Class

The `Reservation` class maintains the available seats. Initially, the system has 10 seats.

The `reserve()` method checks the number of available seats and determines whether the requested booking can be completed.

2. Synchronized Method

The following statement ensures that only one thread at a time can execute the reservation method on the same `Reservation` object.

```
public synchronized void reserve(String personName,
                                 int requestedSeats)
```

When one thread is executing this method, another thread using the same reservation object must wait until the first thread exits the method.

3. Person Class

The `Person` class extends the `Thread` class. Each person represents an individual attempting to reserve seats.

The `run()` method calls the `reserve()` method of the shared reservation object.

4. ReservationDemo Class

The `ReservationDemo` class creates one reservation object and three person threads.

- Person-1 requests 5 seats.
- Person-2 requests 2 seats.

- Person-3 requests 4 seats.

The `start()` method starts each thread. The `join()` method ensures that the previous thread completes before the next thread is started.

Compilation and Execution

Step 1: Save the program

Save the source code as:

`ReservationDemo.java`

Step 2: Compile the program

Open the terminal in the directory containing the file and execute:

```
javac ReservationDemo.java
```

Step 3: Run the program

Execute:

```
java ReservationDemo
```

Sample Output

```
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
-------------------------------------------
Person-2 entered.
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
-------------------------------------------
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
Person-3 leaving.
```

```
Alice requested 5 seats.
5 seats reserved for Alice
Available seats: 5
Bob requested 2 seats.
2 seats reserved for Bob
Available seats: 3
Charlie requested 4 seats.
Sorry, Charlie. Not enough seats available.
Reservation process completed.
```

Test Cases

Test Case 1: Successful Reservation of 5 Seats

Initial Available Seats: 10

Requested Seats: 5

Expected Result: 5 seats reserved successfully.

Remaining Seats: 5

Test Case 2: Successful Reservation of 2 Seats

Initial Available Seats: 5

Requested Seats: 2

Expected Result: 2 seats reserved successfully.

Remaining Seats: 3

Test Case 3: Insufficient Seats

Initial Available Seats: 3

Requested Seats: 4

Expected Result: Requested seats not available.

Remaining Seats: 3

Test Case Summary

Test Case	Available Seats	Requested Seats	Expected Result	Remaining Seats
1	10	5	Booking successful	5
2	5	2	Booking successful	3
3	3	4	Booking unsuccessful	3

Result

Thus, the Java program successfully implements a seat reservation system using multithreading and synchronization. Multiple person threads attempt to reserve seats, and the synchronized `reserve()` method ensures that only one thread at a time can access the reservation system.

Conclusion

The program demonstrates multithreading in Java by creating multiple person threads to book seats. It uses the `synchronized` keyword to prevent simultaneous access to the shared reservation method and maintain the correct seat count. It also demonstrates the use of the `Thread` class, `start()`, `run()`, and `join()` methods.